depending on our requirements. We can create multiple types in schema definitions using the `fields` object inside the main field definition. It has the following syntax:

```
"my_field1":{
  "type": "text",
  "fields": {
    "kw":{ "type":"keyword" }
  }
}
```

Declares the type of my_field1

Defines a fields object to enclose more types

Declares an additional field with the label kw

Basically, `my_field1` is indexed as a `text` type as well as a `keyword` type. When we expect to use it as `text`, we can provide the field as `my_field1` in queries. We use the label `my_fields1.kw` (note the dot notation) as the field name when searching it as a `keyword`.

The following example schema definition creates our single field `subject` with multiple data types (`text`, `keyword`, and `completion`).

Listing 4.43 Schema definition with a multi-typed field

```
PUT emails_multi_type
{
  "mappings": {
    "properties": {
      "subject":{
        "type": "text",
        "fields": {
          "kw":{ "type":"keyword" },
          "comp":{ "type":"completion" }
        }
      }
    }
  }
}
```

The text type

The subject is also a keyword.

The subject is a completion type, too.

The `subject` field has three types associated with it: `text`, `keyword`, and `completion`. To access these, we have to use the format `subject.kw` for the `keyword`-type field or `subject.comp` for the completion type.

We have learned a lot about mapping concepts in this chapter. Let's wrap up and look forward to the next chapter, which discusses working with documents.

Summary

- Every document consists of fields with values, and each field has a data type. Elasticsearch provides a rich set of data types to represent these values.
- Elasticsearch consults a set of rules when indexing and searching data. These rules, called mapping rules, let Elasticsearch know how to deal with the varied data shapes.
- Mapping rules are formed by either dynamic or explicit mapping processes.